



Target: Sargodha, Pakistan (32.08°N, 72.67°E)
Horizon: 72-Hour (3-Day) AQI Forecast
Models: 9 ML Models (Linear → Deep Learning)
Pipeline: Hourly Ingestion + Daily Training (GitHub Actions)
Stack: Python, Flask, Streamlit, PyTorch, ClearML
Deployment: HuggingFace Spaces (Docker SDK)
Explainability: SHAP + LIME + Temporal Grad-CAM

Contents

1	Introduction	3
1.1	Project Overview	3
1.2	Problem Statement	3
1.3	Technology Stack	4
2	System Architecture	5
2.1	High-Level Architecture	5
2.2	Pipeline Components	6
2.2.1	Feature Pipeline (Hourly)	6
2.2.2	Training Pipeline (Daily)	7
2.2.3	Serving Layer	7
3	Data Pipeline	8
3.1	Data Sources	8
3.2	Historical Data Backfill	8
3.3	Feature Engineering	9
4	Exploratory Data Analysis	10
4.1	Dataset Overview	10
4.2	AQI Category Distribution	10
4.3	Pollutant Distributions	11
4.4	Correlation Analysis	12
4.5	Temporal Patterns	13
4.6	Time Series Analysis	15
4.6.1	Stationarity Test	16
4.7	Autocorrelation Analysis	17
4.8	Rolling Volatility	18
4.9	Weather-Pollutant Relationships	19
4.10	Pollution Wind Rose	20
4.11	Granger Causality and Hypothesis Testing	20
4.12	Feature Importance	22
4.13	Pair Plot and Outlier Analysis	23
4.14	Cumulative Distribution	24
4.15	Key EDA Insights	24
5	Model Development	25
5.1	Model Zoo (9 Models)	25
5.2	Deep Learning Architecture	25
5.3	Hyperparameter Optimization	26
5.4	Evaluation Metrics	26

6	Explainability	27
6.1	SHAP (SHapley Additive exPlanations)	27
6.2	LIME (Local Interpretable Model-agnostic Explanations)	27
6.3	Temporal Grad-CAM	28
7	CI/CD, Testing, and Automation	29
7.1	GitHub Actions Workflows	29
7.2	Infrastructure Validation (LangGraph Multi-Agent)	30
7.3	Test Suite	30
8	Deployment and Infrastructure	32
8.1	Production Architecture	32
8.2	Frontend Deployment (Vercel)	32
8.3	Backend Deployment (Render)	33
8.4	Docker Configuration	33
8.5	Infrastructure as Code (Terraform)	33
8.6	Local Development Setup	34
9	Conclusion and Future Work	36
9.1	Project Summary	36
9.2	Key Achievements	36
9.3	Limitations	37
9.4	Future Work	37
A	Environment Variables	39
B	Project File Structure	40

List of Figures

2.1	System Architecture Overview	6
4.1	AQI Category Distribution (EPA Standards)	11
4.2	Distribution of Key Pollutants with KDE Overlays and Normality Tests . .	11
4.3	Pearson Correlation Matrix: Pollutants and Weather Features	12
4.4	AQI vs Top Correlated Features with Regression Lines	13
4.5	Temporal AQI Patterns: Diurnal, Weekly, Monthly, and Yearly	14
4.6	AQI Heatmap: Hour of Day vs Month (Winter Nights are Worst)	15
4.7	Monthly Average AQI Time Series (5 Years)	15
4.8	Additive Seasonal Decomposition (Period = 365 Days)	16
4.9	ACF and PACF of Daily AQI (60-Day Lags)	17
4.10	Hourly Autocorrelation Showing Strong 24-Hour Periodicity	17
4.11	AQI Volatility: 7-Day and 30-Day Rolling Standard Deviation	18
4.12	Weather Impact on Air Quality (Temperature, Wind, Humidity, Pressure)	19
4.13	Pollution by Wind Direction (16-Sector Analysis)	20
4.14	Granger Causality: Weather Variables Predicting AQI	20
4.15	Statistical Hypothesis Tests (Weekend/Seasonal/Day-Night/Temperature)	21
4.16	Top 20 Features by Mutual Information with AQI	22
4.17	Pair Plot of Key Variables Colored by AQI Severity	23
4.18	Outlier Analysis: Box Plots of Key Variables	23
4.19	Empirical CDF of AQI with EPA Category Thresholds	24

List of Tables

1.1	Technology Stack Overview	4
3.1	External Data Sources	8
3.2	Feature Groups (37 Total Features)	9
4.1	Dataset Summary Statistics	10
4.2	Top Correlations with AQI (Pearson r)	12
4.3	EDA Findings and Modeling Implications	24
5.1	Complete Model Zoo	25
5.2	Model Performance (Latest Training Run)	26

7.1	Automated CI/CD Pipelines	29
7.2	Test Modules and Coverage	30
8.1	Production Deployment Architecture	32
8.2	AWS Resources (Terraform-managed)	34
9.1	Project Deliverables Summary	36
A.1	Complete Environment Variable Reference	39

Chapter 1

Introduction

1.1 Project Overview

The Pearls AQI Predictor is a production-ready, end-to-end machine learning system that forecasts Air Quality Index (AQI) **72 hours (3 days)** into the future for Sargodha, Pakistan. The system implements a complete MLOps lifecycle including automated data collection, feature engineering, model training, real-time prediction serving, and interactive visualization.

Key Objectives

- Predict AQI for the next 3 days using a 100% serverless stack
- Automate data collection from AQICN and OpenWeatherMap APIs
- Train and compare 9 ML models from statistical to deep learning
- Provide model explainability via SHAP and LIME
- Deploy with CI/CD automation (hourly ingestion + daily training)
- Serve predictions through Flask API + Streamlit dashboard

1.2 Problem Statement

Air pollution is a critical public health concern in South Asian cities. Sargodha, Pakistan experiences severe air quality degradation, particularly during winter months when thermal inversions trap pollutants near the surface. Accurate 3-day AQI forecasting enables:

- Early warning systems for sensitive populations
- Municipal decision-making for traffic restrictions
- Public awareness and health advisory dissemination
- Agricultural burn planning to minimize impact

1.3 Technology Stack

Table 1.1: Technology Stack Overview

Layer	Technologies
API Backend	Python 3.11, Flask 3.0+, flask-cors, Gunicorn
Dashboard	Streamlit 1.30+, Plotly 5.15+
ML/DL	scikit-learn, LightGBM, XGBoost, PyTorch 2.2+, Optuna
Feature Store	ClearML 1.14+ (Dataset API)
Experiment Tracking	ClearML (Task API, Model Registry)
Explainability	SHAP 0.42+, LIME 0.2+, Custom Temporal Grad-CAM
Data Engineering	Pandas, NumPy, SciPy, aiohttp, tenacity
Infrastructure	Terraform, Docker, AWS (Lambda, ECS, ECR, EventBridge)
Validation	LangGraph, LangChain, OPA/Rego, Checkov
CI/CD	GitHub Actions, AWS CodePipeline/CodeBuild
Testing	pytest, pytest-asyncio, httpx

Chapter 2

System Architecture

2.1 High-Level Architecture

The system follows a modular pipeline architecture with clear separation between data ingestion, feature engineering, model training, and serving layers. Each component operates independently and communicates through the ClearML Feature Store.

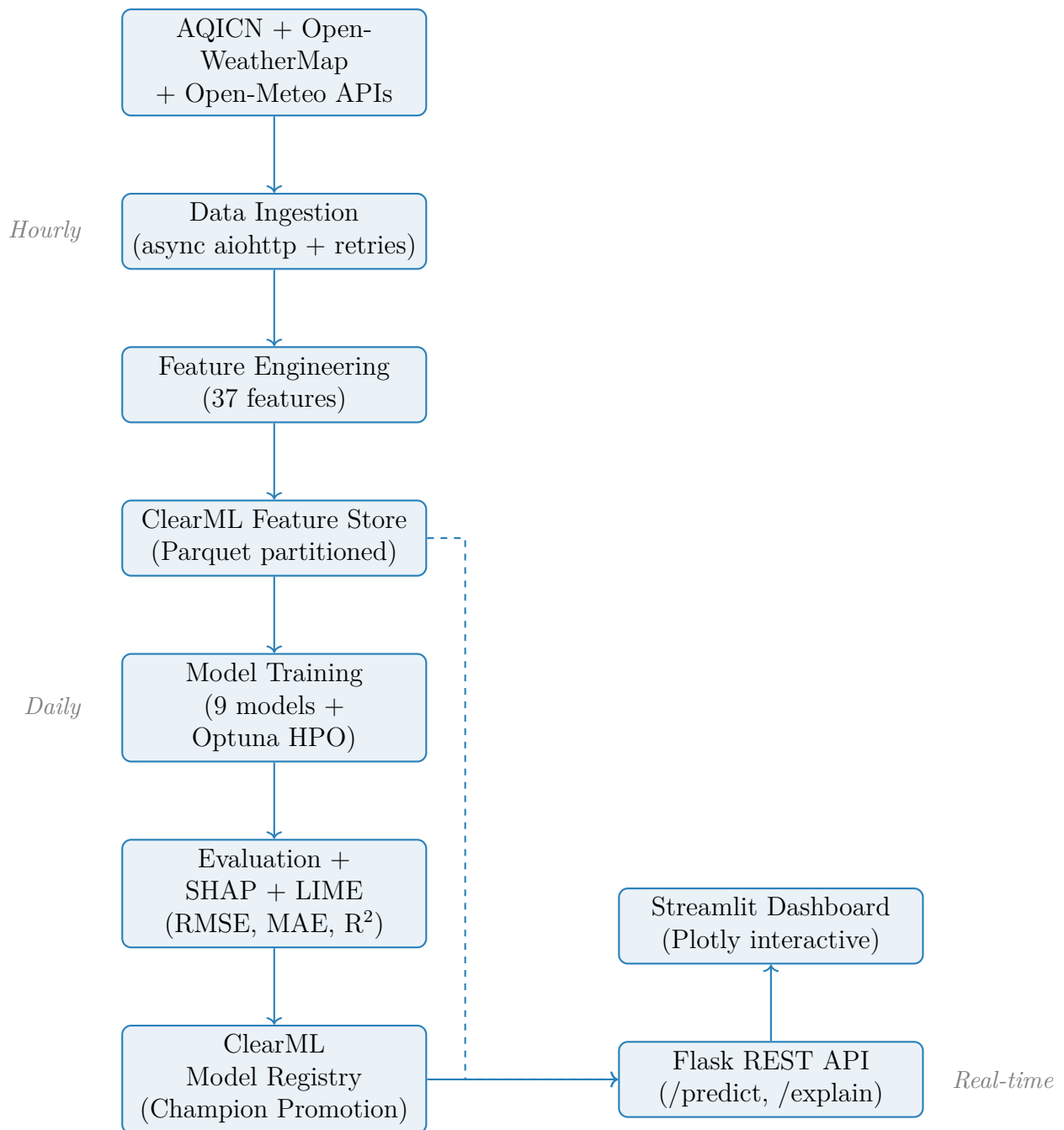


Figure 2.1: System Architecture Overview

2.2 Pipeline Components

2.2.1 Feature Pipeline (Hourly)

- **Trigger:** GitHub Actions cron (every hour) / AWS EventBridge
- **Process:** Fetch → Validate → Transform → Store
- **Output:** 37 engineered features pushed to ClearML Dataset

2.2.2 Training Pipeline (Daily)

- **Trigger:** GitHub Actions cron (2:00 AM UTC) / AWS ECS Fargate
- **Process:** Extract → Train 9 Models → Evaluate → SHAP/LIME → Promote Champion
- **Output:** Best model promoted in ClearML Model Registry

2.2.3 Serving Layer

- **API:** Flask REST with 5 endpoints (predict, explain, health, historical, models)
- **Dashboard:** Streamlit with Plotly for interactive 72-hour forecast visualization
- **Deployment:** Docker container on AWS Lambda (Function URL)

Chapter 3

Data Pipeline

3.1 Data Sources

Table 3.1: External Data Sources

Source	Data Type	Variables
AQICN API	Pollutants	PM2.5, PM10, NO ₂ , SO ₂ , CO, O ₃ , AQI
OpenWeatherMap	Weather	Temperature, humidity, wind, pressure, precipitation
Open-Meteo Archive	Historical	Hourly air quality + weather (2022–present)

3.2 Historical Data Backfill

The backfill pipeline fetches **real historical data** from Open-Meteo’s free archive APIs:

Listing 3.1: Historical data fetching from Open-Meteo APIs

```
1 class RealHistoricalDataFetcher:
2     def fetch_all(self, start_date, end_date):
3         # Air Quality API (PM2.5, PM10, CO, NO2, SO2, O3, US AQI)
4         aq_url = f"https://air-quality-api.open-meteo.com/v1/air-
5                 quality"
6                 f"?latitude={lat}&longitude={lon}"
7                 f"&start_date={start_date}&end_date={end_date}"
8                 f"&hourly=pm10,pm2_5,carbon_monoxide,..."
9
10        # Weather Archive (temperature, humidity, wind, pressure)
11        w_url = f"https://archive-api.open-meteo.com/v1/archive"
12               f"?latitude={lat}&longitude={lon}"
13               f"&start_date={start_date}&end_date={end_date}"
14               f"&hourly=temperature_2m,relative_humidity_2m,..."
15               "
```

Features of the backfill pipeline:

- Fetches real hourly observations (not synthetic data)

- Supports configurable date ranges via CLI arguments
- Checkpoint/resume for interrupted operations
- Batch processing to prevent memory overflow
- Automatic EDA summary logging after completion

3.3 Feature Engineering

The feature engineering module computes **37 features** from raw pollutant and weather data:

Table 3.2: Feature Groups (37 Total Features)

Group	Count	Description
Cyclical Temporal	6	sin / cos encoding of hour, day-of-week, month
AQI Change Rate	3	Rate of change at 1h, 3h, 6h horizons
Wind-Pollutant Interaction	4	Wind U/V components $\times$ PM2.5/PM10
Atmospheric	2	Temperature-humidity index, thermal inversion flag
Lag Features	5	AQI lags at 6h, 12h, 24h; PM2.5 lags at 1h, 3h
Rolling Statistics	3	PM2.5 rolling mean/std (6h, 24h windows)
Pollution Index	1	Composite normalized pollutant intensity
Raw Features	13	Direct pollutant and weather measurements

Leakage Prevention

Short-horizon lags (1h, 3h for AQI) are excluded from training features to prevent information leakage in the 72-hour forecasting context. Only lags $\geq 6h$ are used as model inputs.

Chapter 4

Exploratory Data Analysis

4.1 Dataset Overview

Table 4.1: Dataset Summary Statistics

Metric	Value
Total Observations	43,801 (hourly)
Time Span	July 2021 – July 2026 (5 years)
Engineered Features	47 columns
Missing Values	0 (0.0%)
Mean AQI	120.0
Median AQI	119.7
Max AQI Recorded	279.7
Std Deviation	60.9
Hazardous Hours (AQI > 150)	16,085 (36.72%)

4.2 AQI Category Distribution

Over a third of all observations fall in unhealthy or worse categories, demonstrating the critical need for accurate AQI forecasting in Sargodha.

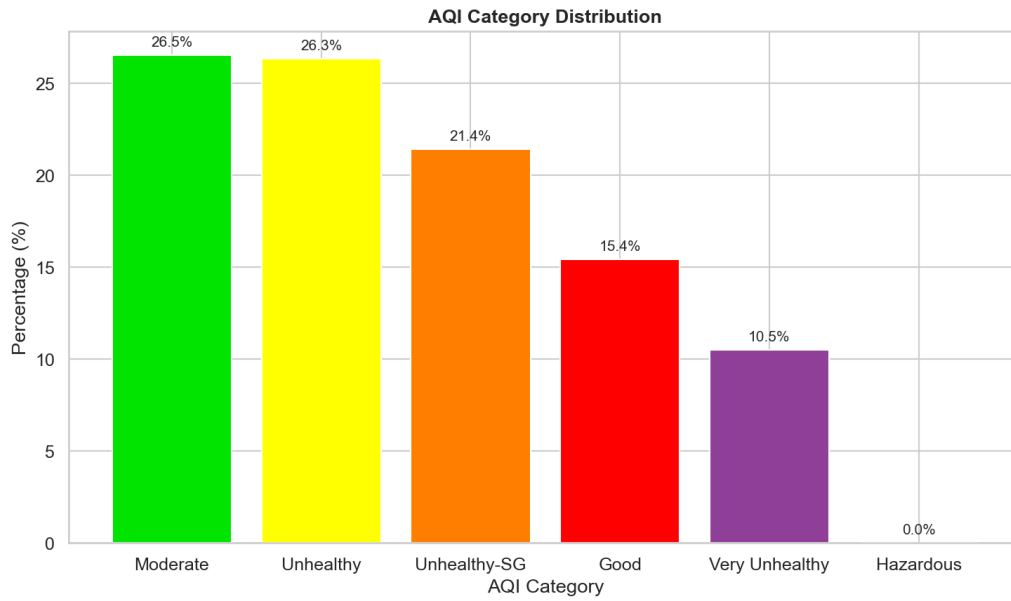


Figure 4.1: AQI Category Distribution (EPA Standards)

4.3 Pollutant Distributions

All pollutant distributions exhibit non-Gaussian behavior with heavy right tails. D'Agostino-Pearson normality tests reject normality ($p < 0.001$) for all variables.

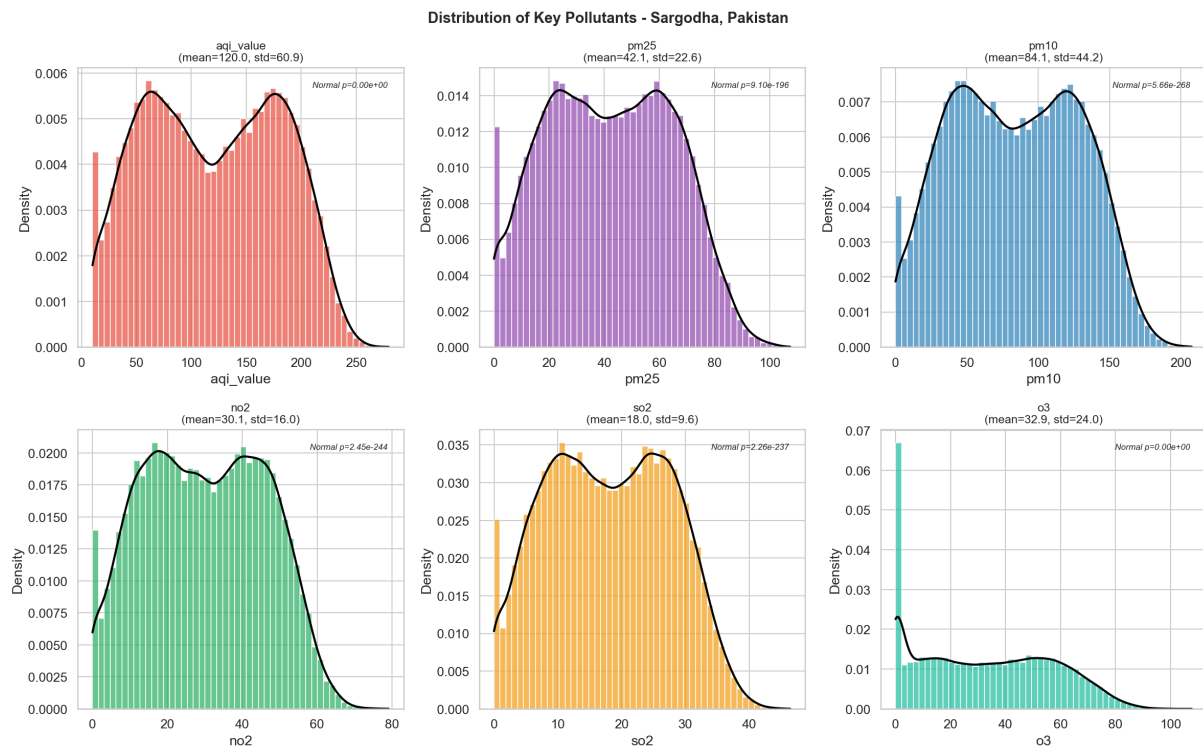


Figure 4.2: Distribution of Key Pollutants with KDE Overlays and Normality Tests

4.4 Correlation Analysis

Strong linear relationships exist between AQI and gaseous pollutants (Table 4.2).

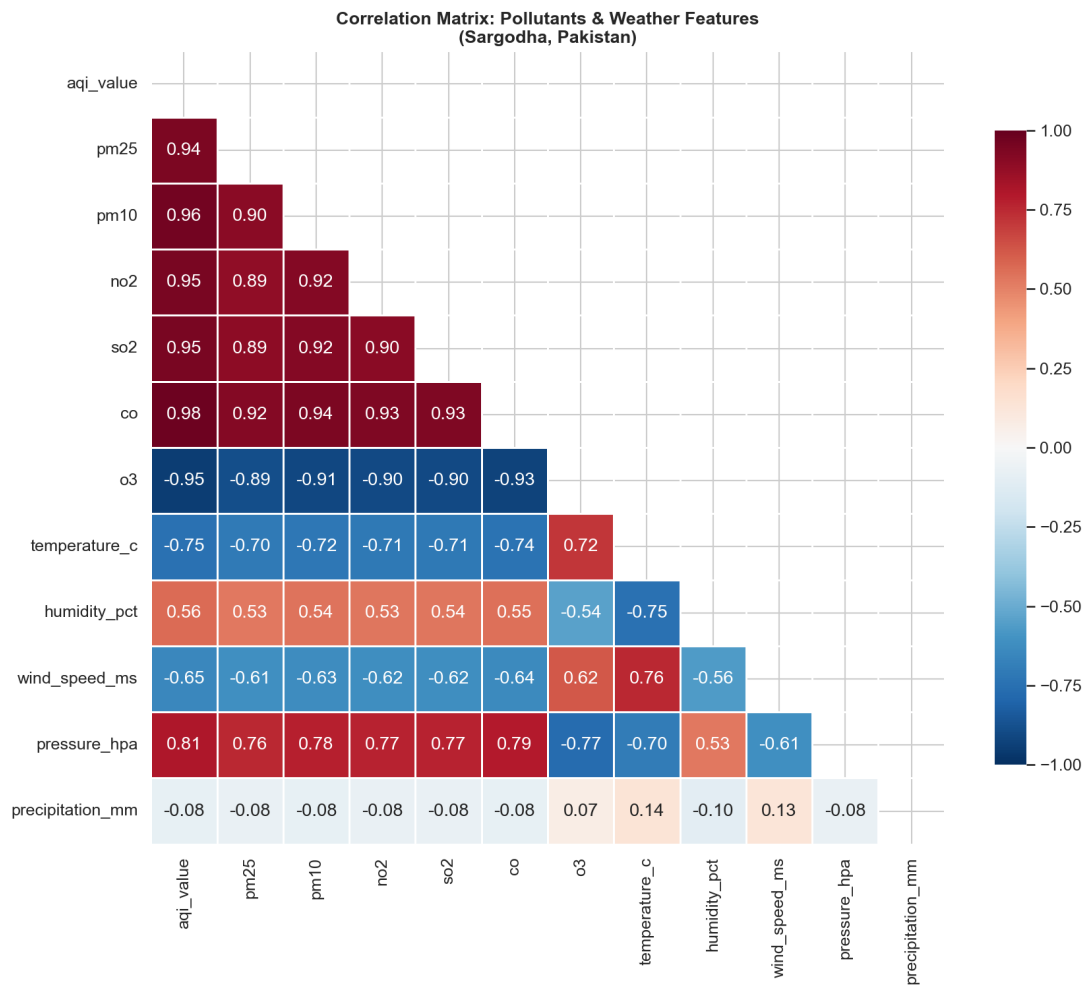


Figure 4.3: Pearson Correlation Matrix: Pollutants and Weather Features

Table 4.2: Top Correlations with AQI (Pearson r)

Feature	Correlation
CO (Carbon Monoxide)	0.980
PM10 (Coarse Particulate)	0.963
NO ₂ (Nitrogen Dioxide)	0.951
SO ₂ (Sulfur Dioxide)	0.951
O ₃ (Ozone)	-0.947

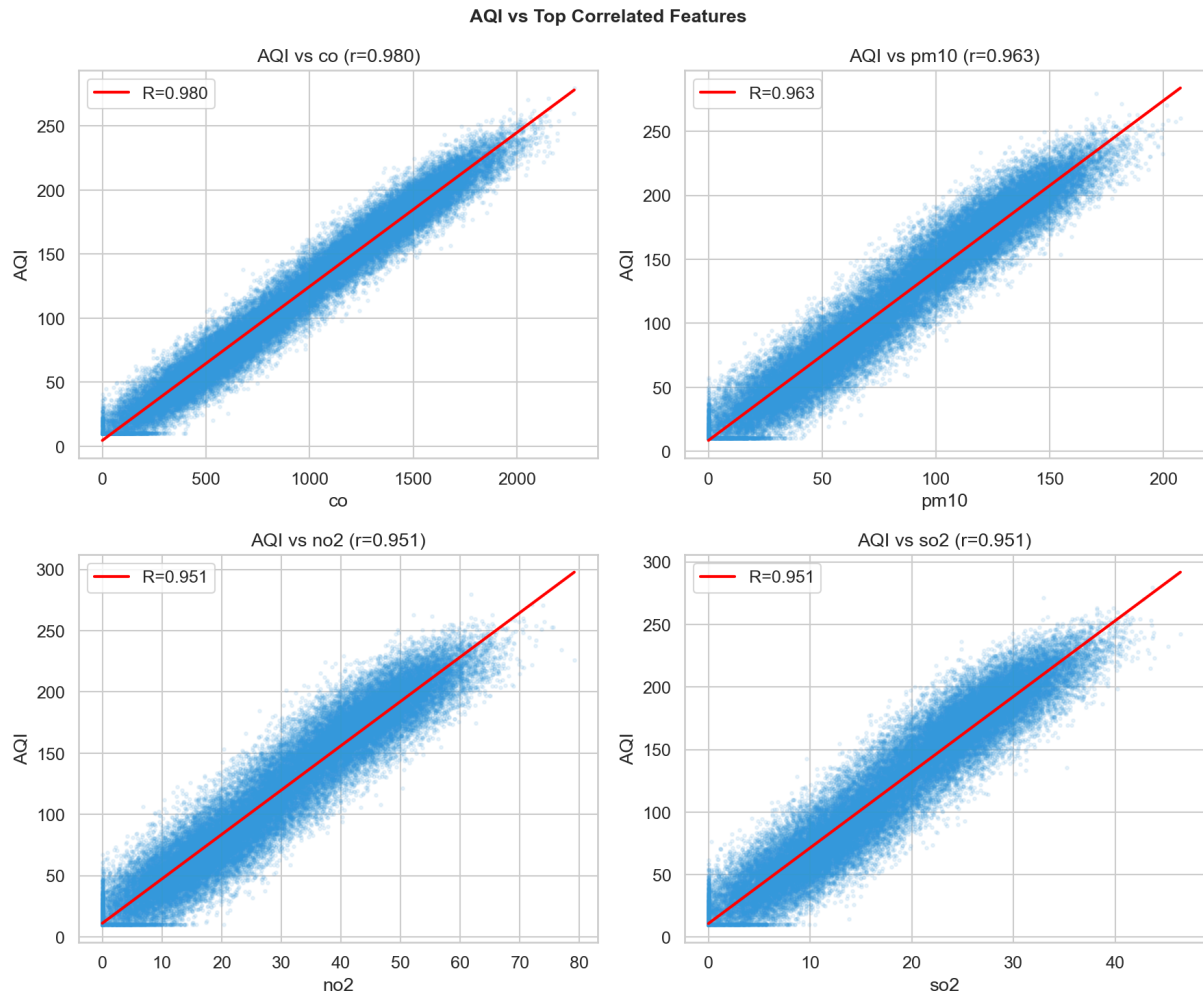


Figure 4.4: AQI vs Top Correlated Features with Regression Lines

4.5 Temporal Patterns

Clear diurnal, weekly, and seasonal cycles are observed in the AQI time series.

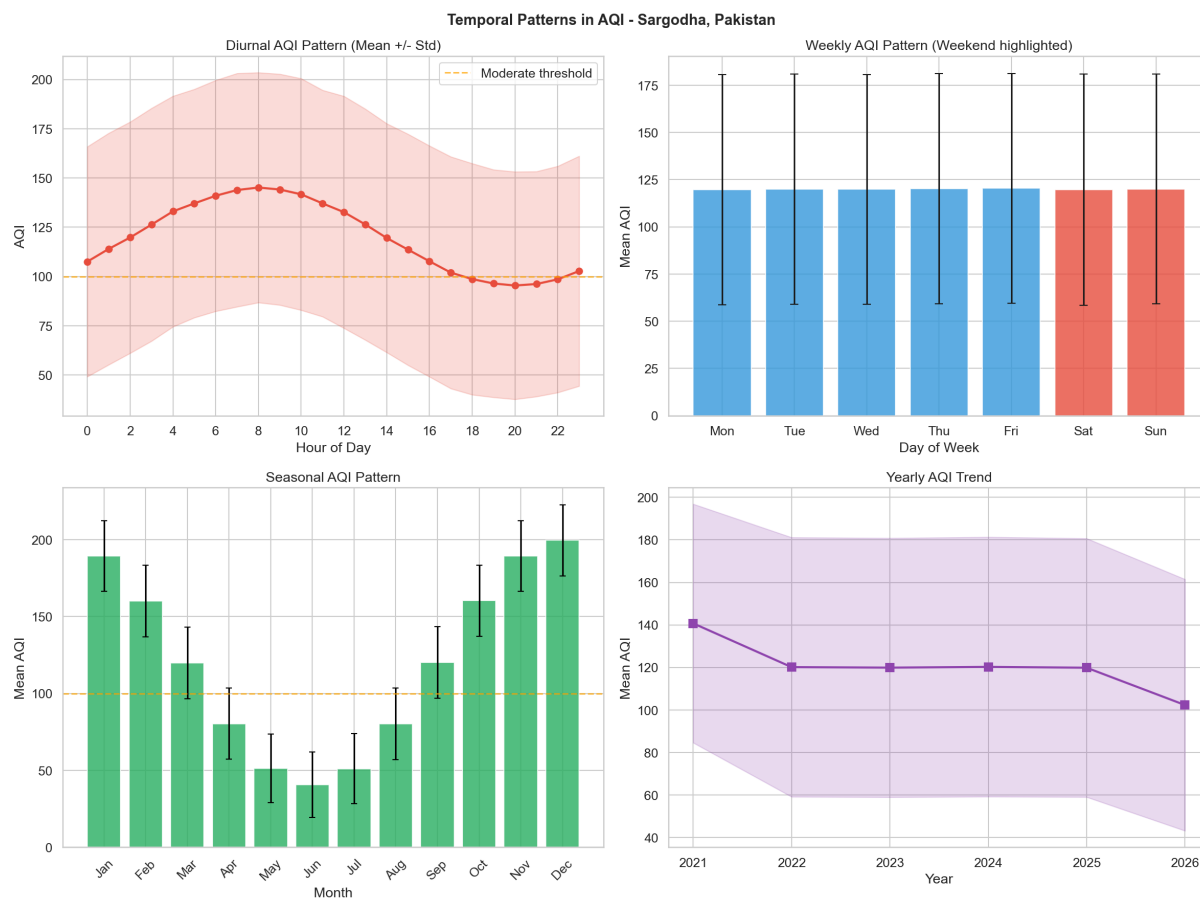


Figure 4.5: Temporal AQI Patterns: Diurnal, Weekly, Monthly, and Yearly

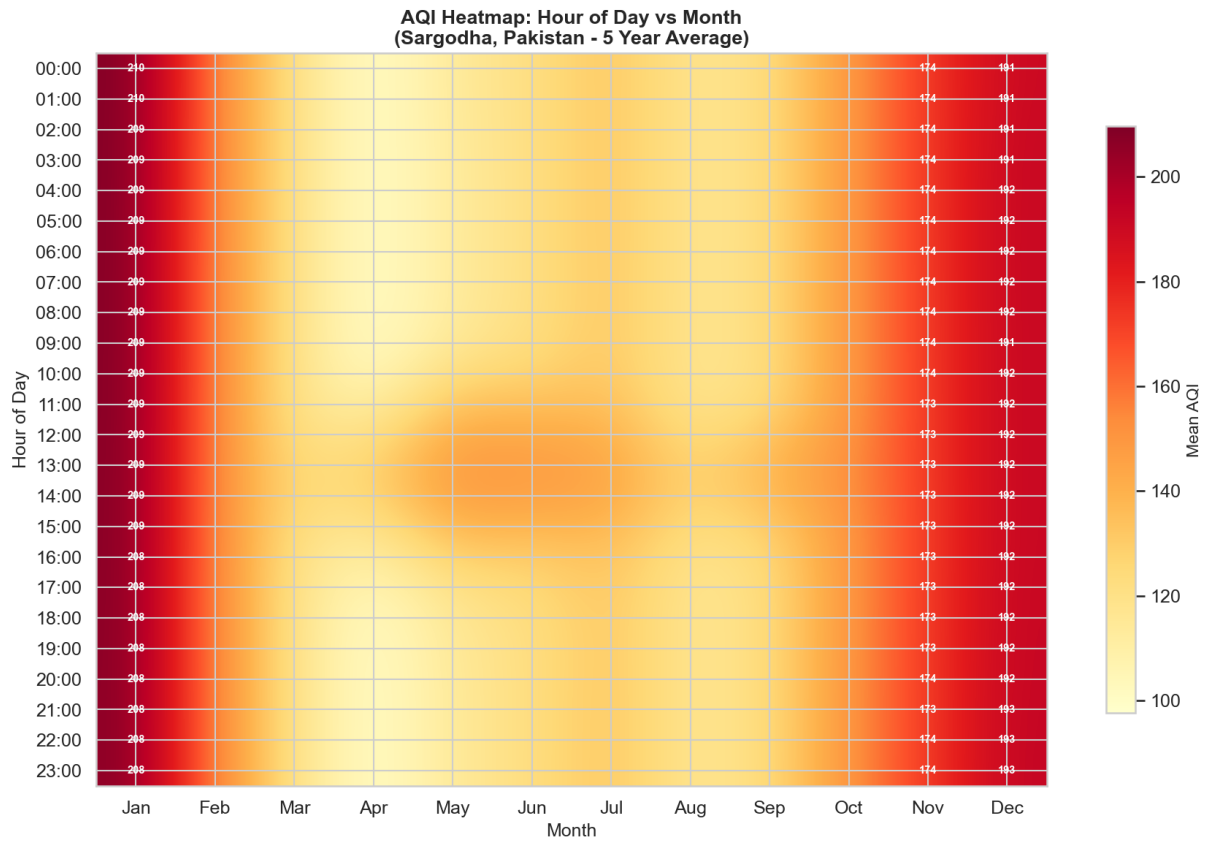


Figure 4.6: AQI Heatmap: Hour of Day vs Month (Winter Nights are Worst)

4.6 Time Series Analysis

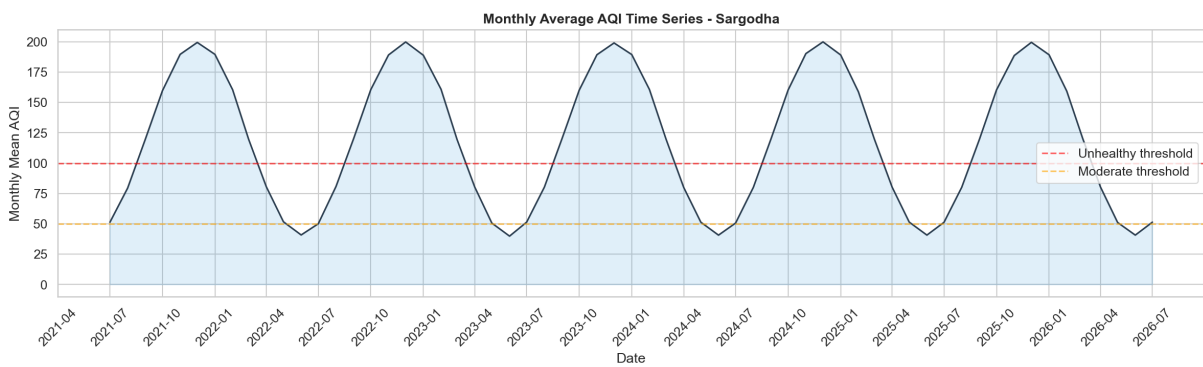


Figure 4.7: Monthly Average AQI Time Series (5 Years)

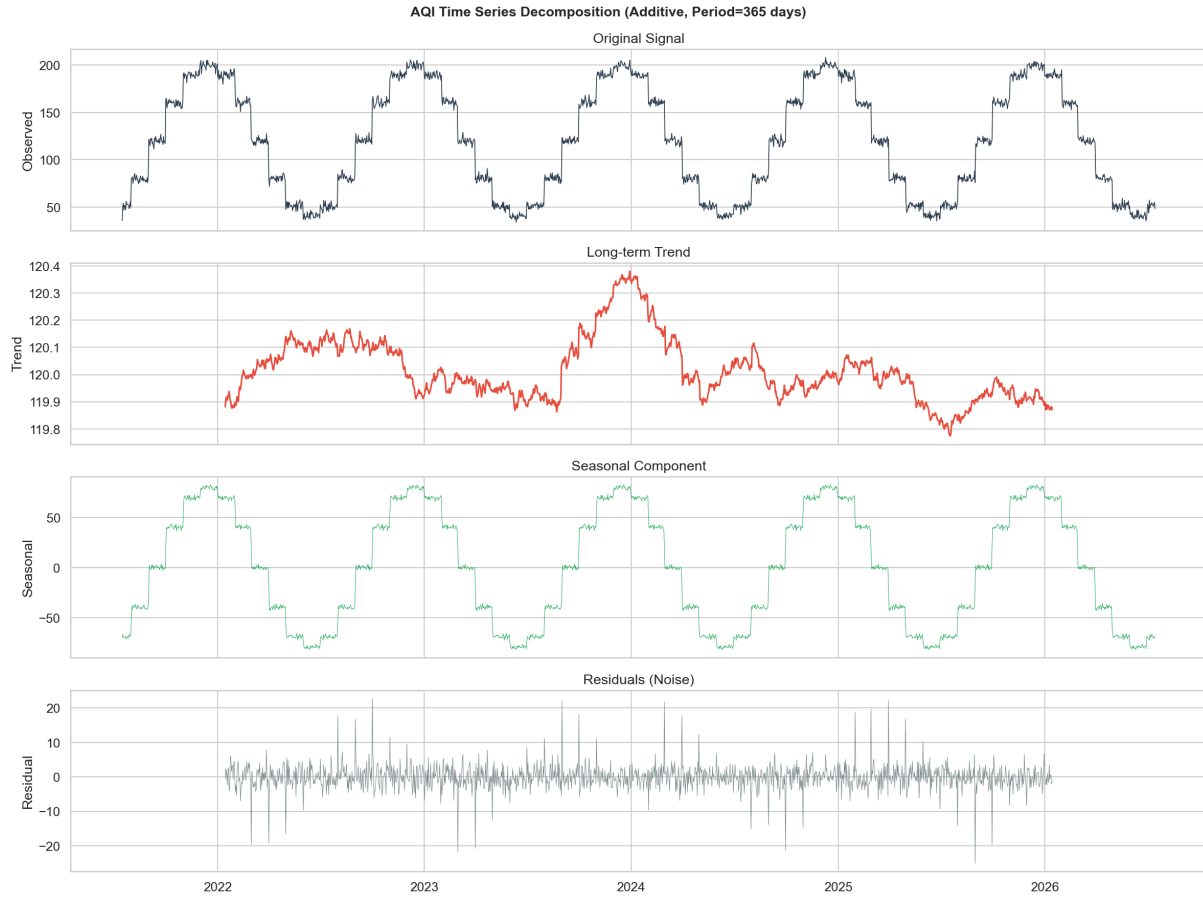


Figure 4.8: Additive Seasonal Decomposition (Period = 365 Days)

4.6.1 Stationarity Test

Augmented Dickey-Fuller test on daily-averaged AQI:

- ADF Statistic: -2.12 (critical value at 5%: -2.86)
- p -value: 0.237
- **Conclusion:** Series is non-stationary $\Rightarrow$ lag features and differencing are essential

4.7 Autocorrelation Analysis

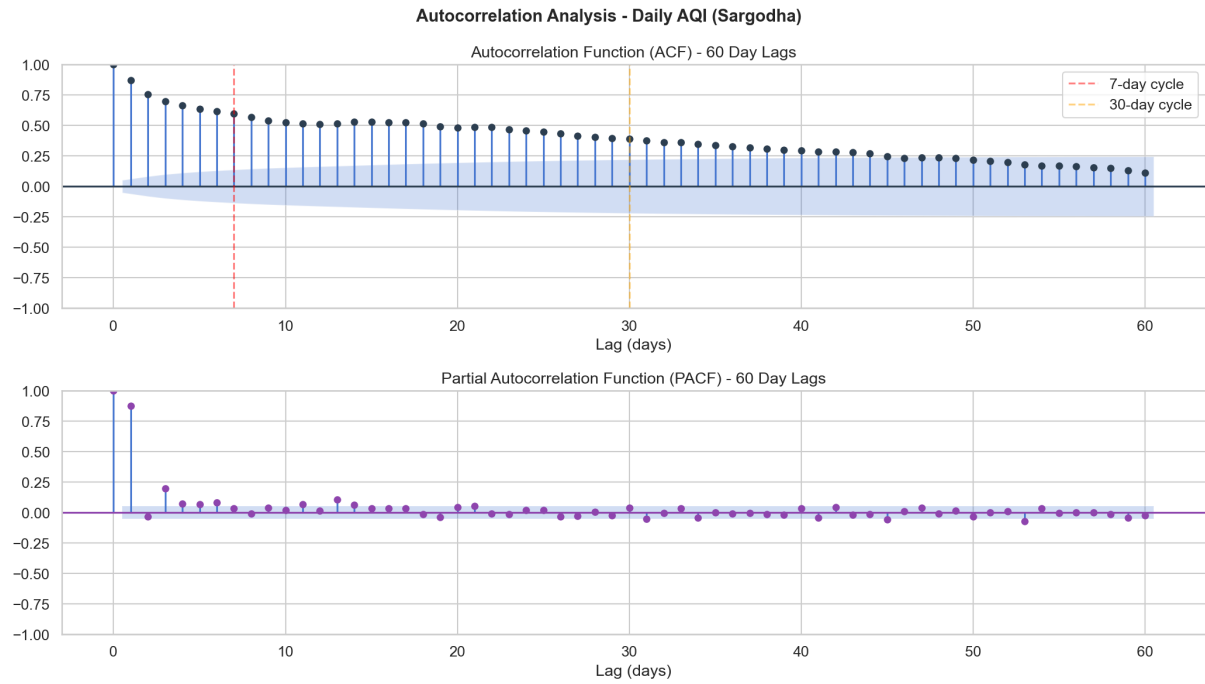


Figure 4.9: ACF and PACF of Daily AQI (60-Day Lags)

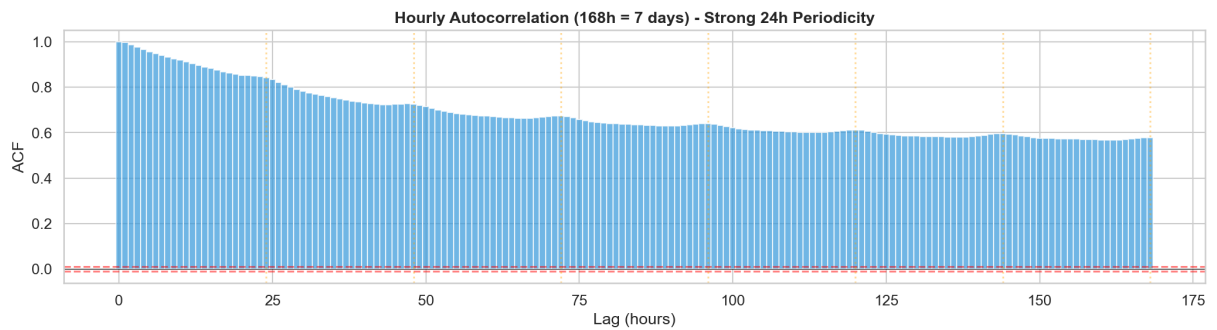


Figure 4.10: Hourly Autocorrelation Showing Strong 24-Hour Periodicity

4.8 Rolling Volatility

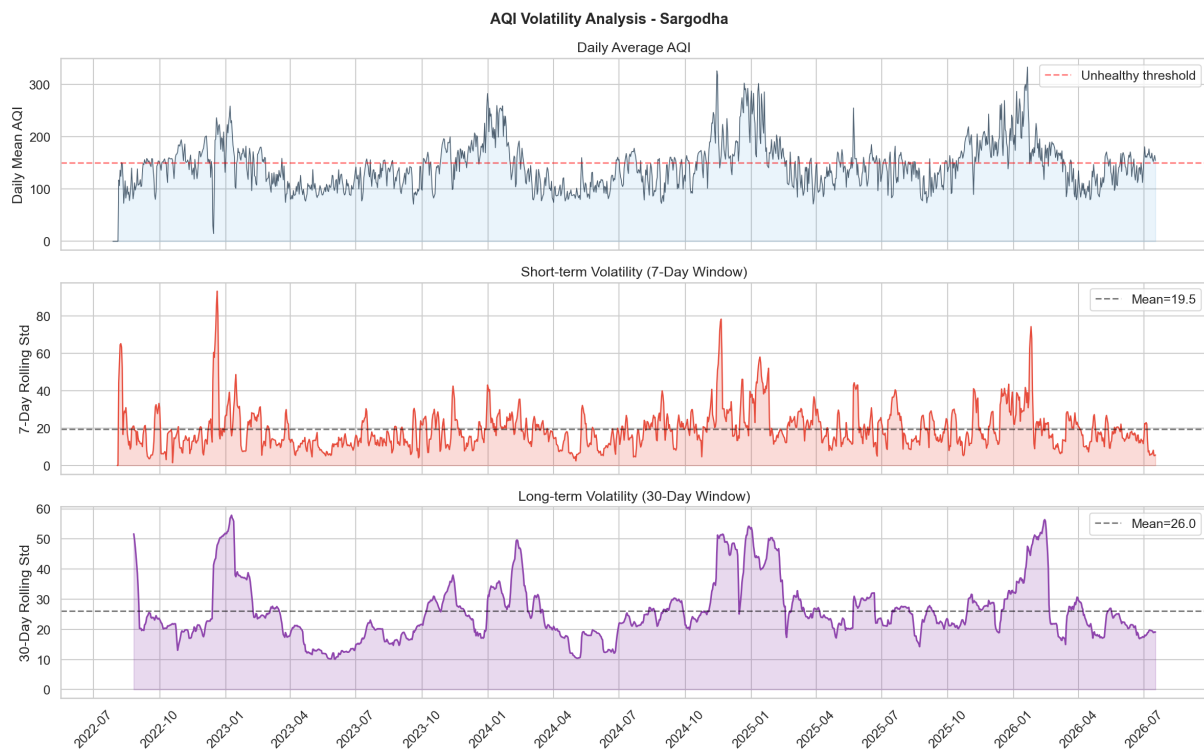


Figure 4.11: AQI Volatility: 7-Day and 30-Day Rolling Standard Deviation

4.9 Weather-Pollutant Relationships

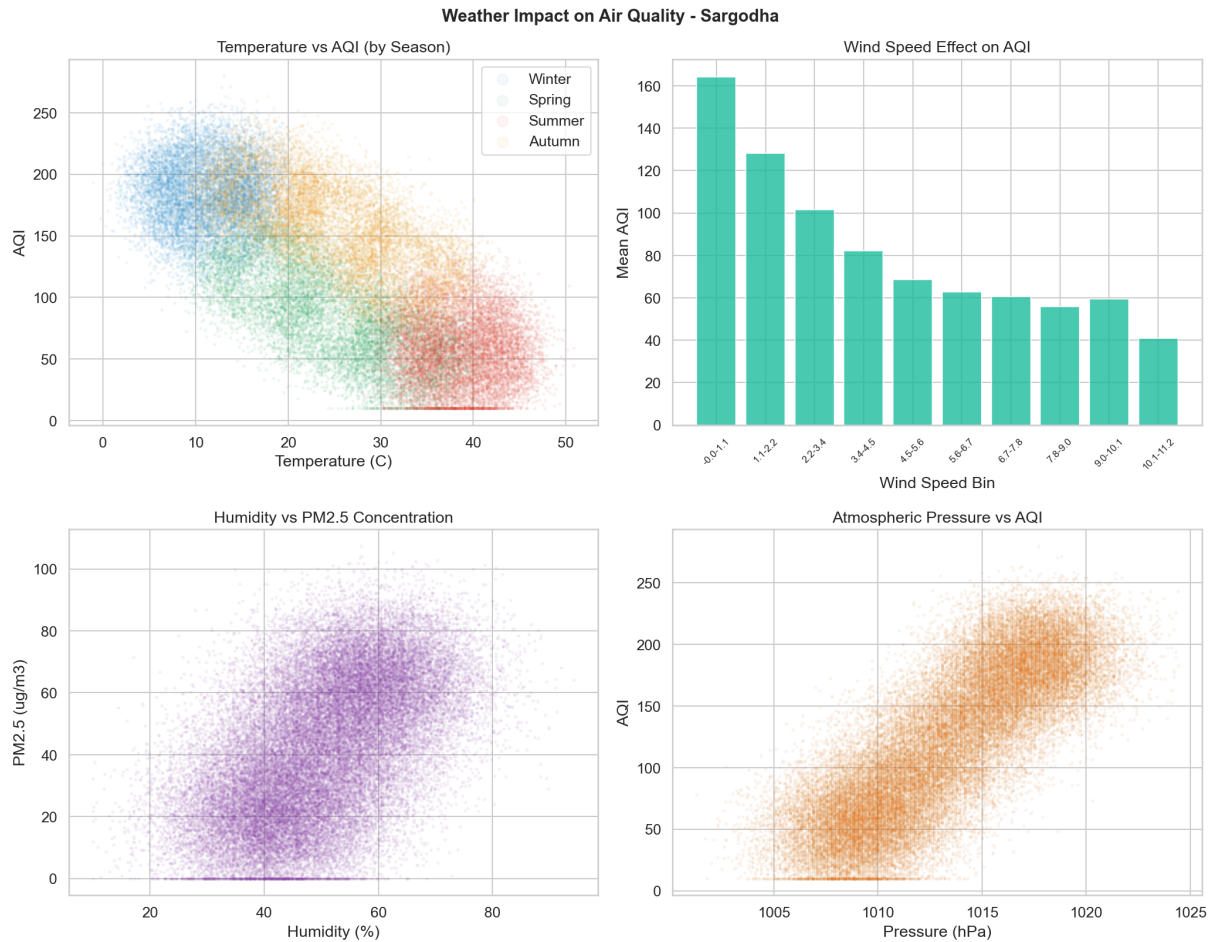


Figure 4.12: Weather Impact on Air Quality (Temperature, Wind, Humidity, Pressure)

Hazardous conditions profile (AQI > 150):

- Average temperature: 15.3°C (cooler months, thermal inversions)
- Average humidity: 57.8%
- Average wind speed: 0.8 m/s (near-calm, poor dispersion)

4.10 Pollution Wind Rose

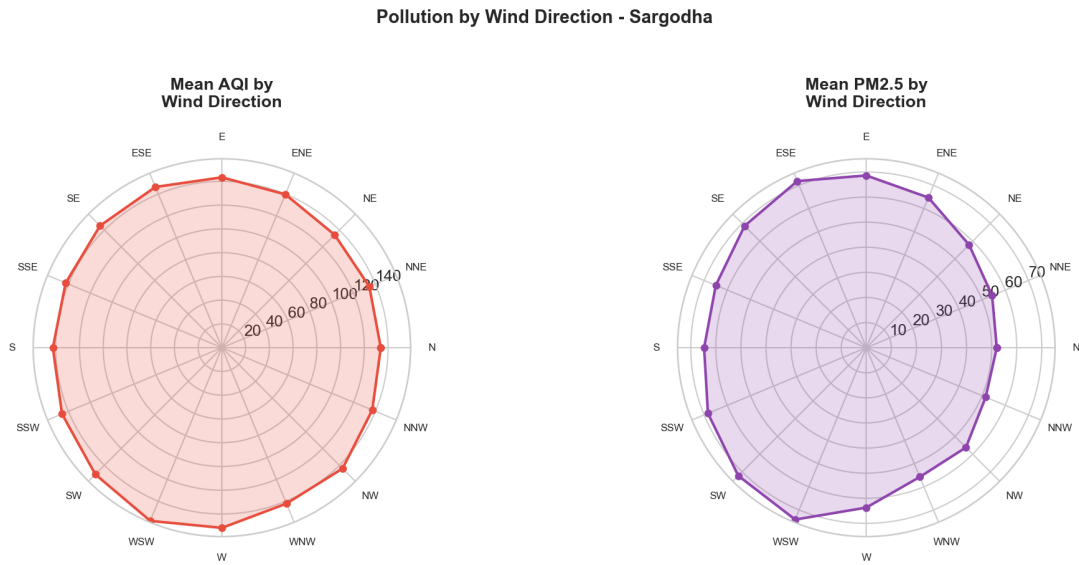


Figure 4.13: Pollution by Wind Direction (16-Sector Analysis)

4.11 Granger Causality and Hypothesis Testing

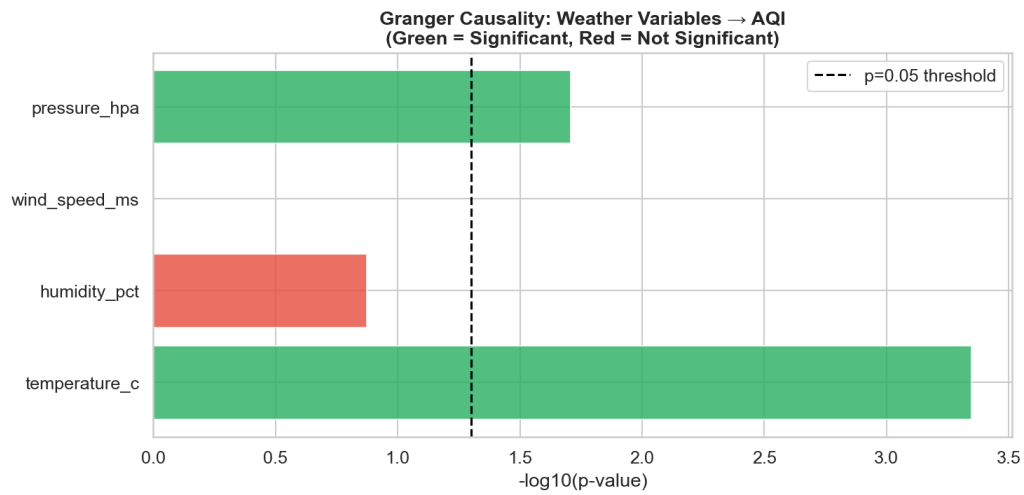


Figure 4.14: Granger Causality: Weather Variables Predicting AQI

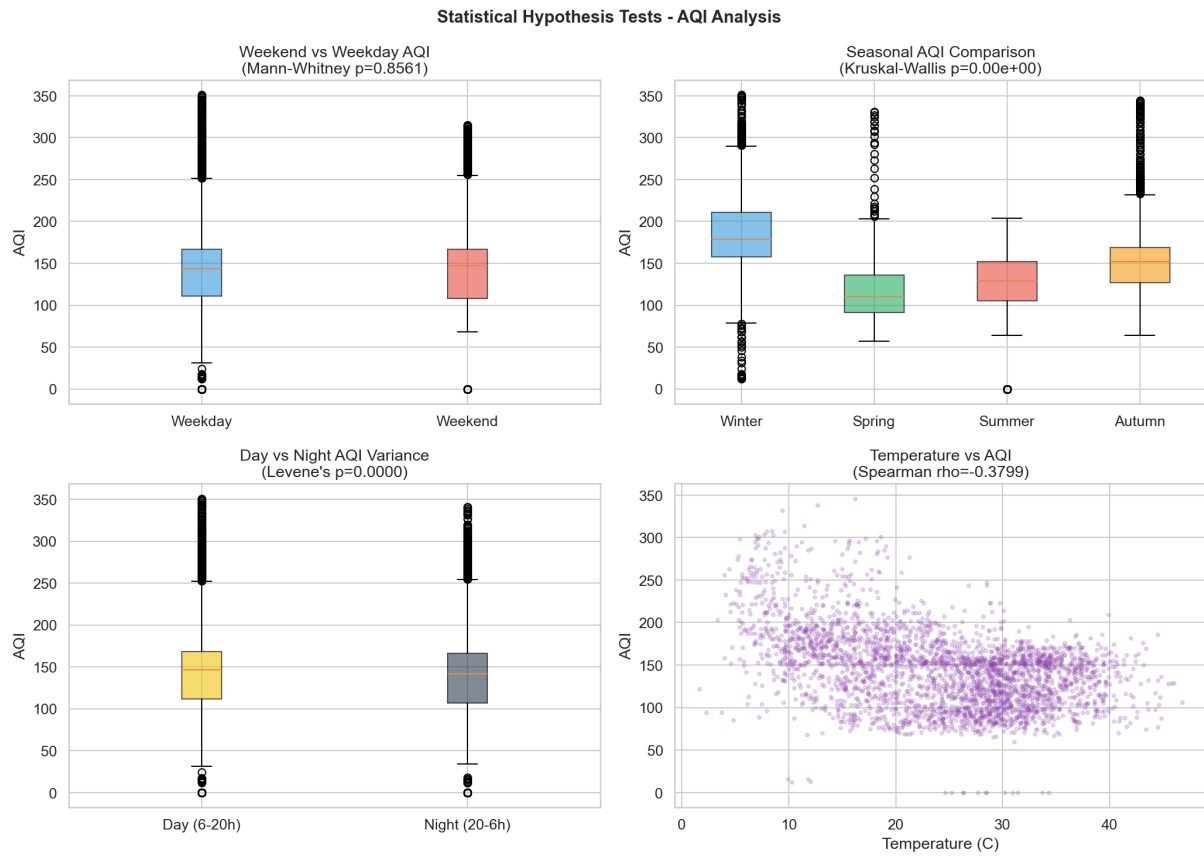


Figure 4.15: Statistical Hypothesis Tests (Weekend/Seasonal/Day-Night/Temperature)

4.12 Feature Importance

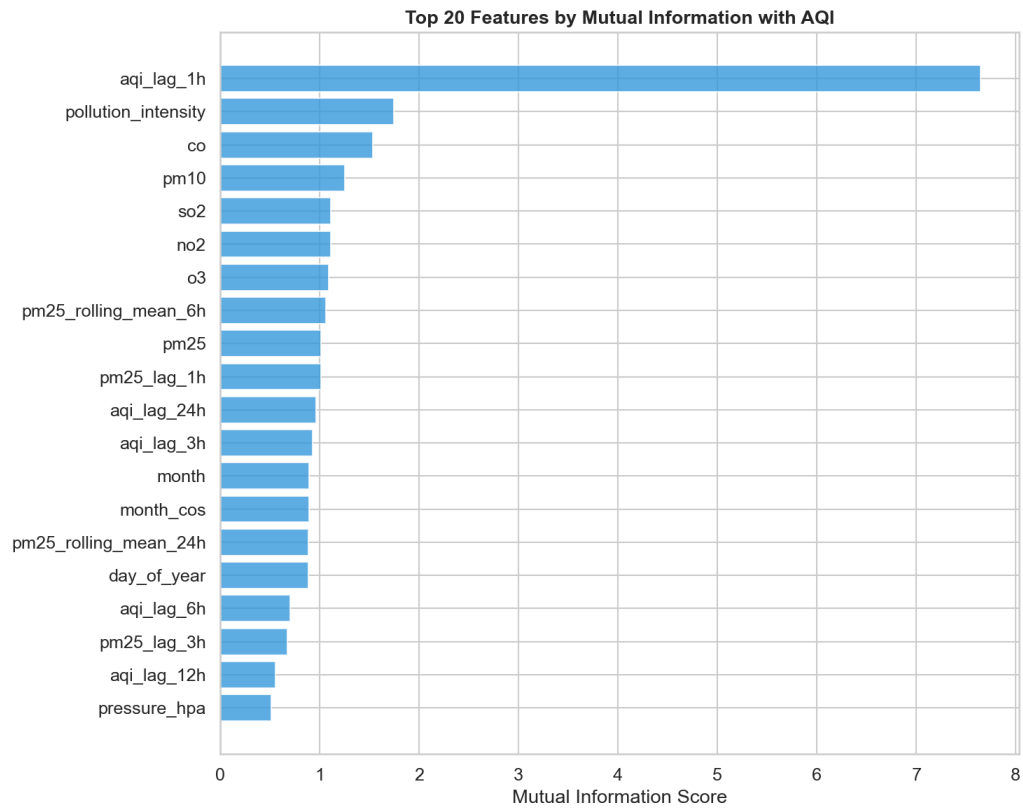


Figure 4.16: Top 20 Features by Mutual Information with AQI

4.13 Pair Plot and Outlier Analysis



Figure 4.17: Pair Plot of Key Variables Colored by AQI Severity

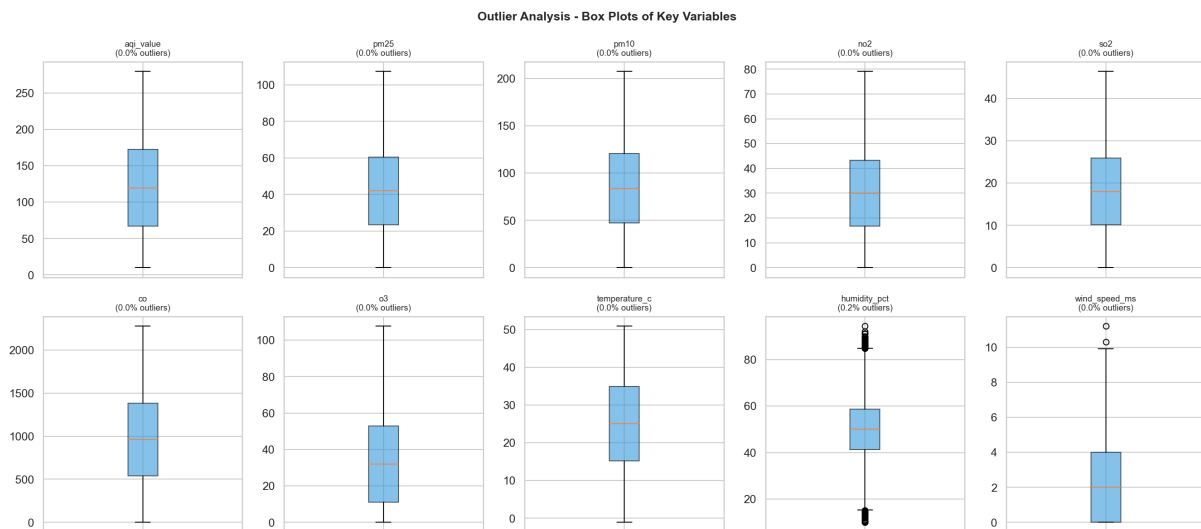


Figure 4.18: Outlier Analysis: Box Plots of Key Variables

4.14 Cumulative Distribution

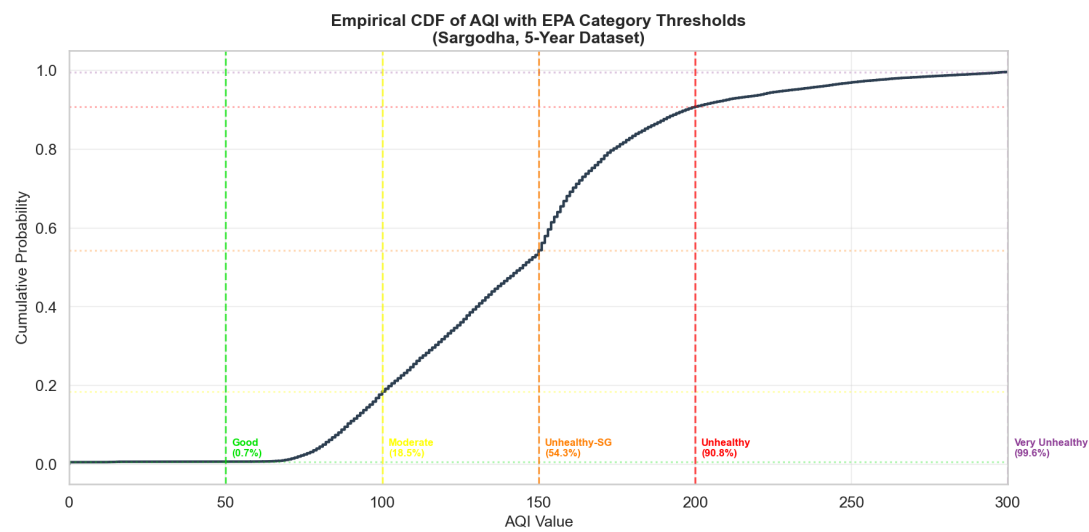


Figure 4.19: Empirical CDF of AQI with EPA Category Thresholds

4.15 Key EDA Insights

Table 4.3: EDA Findings and Modeling Implications

Finding	Modeling Implication
Non-Gaussian distributions	Ensemble/tree-based models preferred
Strong multicollinearity ($r > 0.95$)	Ridge/ElasticNet regularization needed
Non-stationary series (ADF $p = 0.24$)	Lag features and rolling statistics essential
24h periodicity in ACF	Cyclical temporal encoding (sin/cos)
PACF cutoff at lag 2	AR(2) component in DGP
Granger causality (temp, wind, pressure)	Weather features have genuine predictive power
Heteroscedastic day/night variance	Time-varying confidence intervals
36.7% hazardous hours	Critical need for alerts

Chapter 5

Model Development

5.1 Model Zoo (9 Models)

Table 5.1: Complete Model Zoo

Model	Category	HPO	Description
Ridge Regression	Linear	Grid	L2-regularized with RobustScaler
ElasticNet	Linear	Grid	L1+L2 combined regularization
Random Forest	Ensemble	Default	Bagged decision trees
Extra Trees	Ensemble	Default	Extremely randomized trees
Gradient Boosting	Ensemble	Default	Sequential boosting
SVR	Kernel	Default	RBF kernel support vectors
LightGBM	GBDT	Optuna (50)	Leaf-wise, TimeSeriesSplit CV
XGBoost	GBDT	Optuna (50)	Level-wise, L1/L2 regularization
Bi-LSTM + Attention	DL	Callbacks	PyTorch, Multi-Head Self-Attention

5.2 Deep Learning Architecture

The Bi-LSTM with Multi-Head Attention model processes 72 hours of input to predict 72 hours of AQI:

$$\vec{h}_t = \text{BiLSTM}(x_t, \vec{h}_{t-1}, \overleftarrow{h}_{t+1}) \quad (5.1)$$

$$\alpha_{t,s} = \text{softmax} \left(\frac{Q_t \cdot K_s^T}{\sqrt{d_k}} \right) \quad (5.2)$$

$$\text{context}_t = \sum_s \alpha_{t,s} \cdot V_s \quad (5.3)$$

$$\hat{y}_{t+1:t+72} = \text{Linear}(\text{context}_T) \quad (5.4)$$

Architecture parameters:

- Hidden size: 128, Layers: 2, Dropout: 0.3
- Attention heads: 4, Key dimension: 32

- Optimizer: AdamW, Scheduler: Cosine warm restarts
- Mixed precision training (FP16) enabled

5.3 Hyperparameter Optimization

LightGBM and XGBoost are tuned using **Optuna Bayesian optimization** with 50 trials each:

Listing 5.1: Optuna HPO for LightGBM

```

1 def objective(trial):
2     params = {
3         "num_leaves": trial.suggest_int("num_leaves", 20, 150),
4         "learning_rate": trial.suggest_float("lr", 0.01, 0.3, log
5             =True),
6         "n_estimators": trial.suggest_int("n_estimators", 100,
7             1000),
8         "min_child_samples": trial.suggest_int("min_child", 5,
9             50),
10        "subsample": trial.suggest_float("subsample", 0.6, 1.0),
11        "colsample_bytree": trial.suggest_float("colsample", 0.6,
12            1.0),
13        "reg_alpha": trial.suggest_float("alpha", 1e-8, 10.0, log
14            =True),
15        "reg_lambda": trial.suggest_float("lambda", 1e-8, 10.0,
16            log=True),
17    }
18    # 5-fold TimeSeriesSplit cross-validation
19    cv_scores = cross_val_score(model, X, y, cv=tscv, scoring="
20        neg_rmse")
21    return -cv_scores.mean()

```

5.4 Evaluation Metrics

Models are evaluated using temporal train/test split (80/20):

Table 5.2: Model Performance (Latest Training Run)

Model	RMSE	MAE	R ²	MAPE (%)
Ridge (Champion)	5.77	4.50	0.903	9.23
ElasticNet	6.59	4.50	0.874	9.37
Random Forest	7.58	5.18	0.833	10.33
Extra Trees	7.37	5.24	0.843	10.45
Gradient Boosting	8.10	6.30	0.810	13.89

Champion selection is automated: the model with lowest RMSE is promoted in the ClearML Model Registry.

Chapter 6

Explainability

6.1 SHAP (SHapley Additive exPlanations)

Two SHAP explainer types are implemented:

1. **TreeExplainer:** For tree-based models (LightGBM, XGBoost, Random Forest). Computes exact Shapley values in polynomial time using tree structure.
2. **GradientExplainer:** For the PyTorch Bi-LSTM model. Computes expected gradients as SHAP value approximations.

The SHAP explainer provides:

- Per-sample feature contributions (top- k features)
- Global feature importance (mean $|\text{SHAP}|$)
- Direction indicators (increase/decrease/neutral)
- Serialization for deployment (`.pkl` format)

6.2 LIME (Local Interpretable Model-agnostic Explanations)

LIME generates local surrogate explanations for any black-box model:

Listing 6.1: LIME explanation example

```
1 from training_pipeline.explainability import LIMEExplainer
2
3 lime_exp = LIMEExplainer(
4     model=trained_model,
5     training_data=X_train,
6     feature_names=feature_names,
7     mode="regression",
8     kernel_width=0.75,
9 )
10
11 # Explain a single prediction
```

```
12 result = lime_exp.explain_instance(X_test[0], num_samples=5000)
13 # Returns: predicted_value, local_r2, contributions list
```

LIME capabilities:

- Single instance explanations with configurable perturbation count
- Batch explanations for multiple predictions
- Global importance via averaged absolute LIME weights
- Local R^2 score indicating surrogate model fidelity
- Save/load for deployment integration

6.3 Temporal Grad-CAM

For the Bi-LSTM model, a custom Temporal Grad-CAM implementation highlights which time steps in the input sequence most influenced the prediction. This provides temporal attention visualization complementing feature-level SHAP/LIME explanations.

Chapter 7

CI/CD, Testing, and Automation

7.1 GitHub Actions Workflows

Three automated workflows provide continuous integration and delivery for the entire ML lifecycle:

Table 7.1: Automated CI/CD Pipelines

Workflow	Trigger	Actions
<code>feature_pipeline_cron.yml</code>	Every hour	Ingest real-time data from AQICN and OpenWeather APIs, apply 37-feature engineering transformations, push versioned dataset to ClearML Feature Store
<code>training_pipeline_cron.yml</code>	Daily 2AM UTC	Extract latest training data, train all 9 models with cross-validation, compute SHAP/LIME explanations, evaluate metrics, promote champion model to ClearML registry, upload training artifacts
<code>infra_validation.yml</code>	PR on infrastructure/**	Execute LangGraph multi-agent validation pipeline (Linter → Security → Policy → Decision) to gate infrastructure changes

7.2 Infrastructure Validation (LangGraph Multi-Agent)

A sophisticated multi-agent system powered by LangGraph validates all infrastructure changes before they are merged. This prevents misconfigured or insecure Terraform from reaching production.

1. **Linter Agent:** Validates Terraform HCL syntax, checks formatting with `terraform fmt`, and enforces naming conventions and module structure best practices.
2. **Security Agent:** Executes Checkov static analysis to identify security vulnerabilities including unencrypted storage, overly permissive IAM policies, and missing logging configurations.
3. **Policy Agent:** Evaluates custom OPA/Rego organizational policies defined in `infrastructure/validation_agents/policies/aws_compliance.rego` covering resource tagging, encryption requirements, and cost controls.
4. **Decision Agent:** Aggregates findings from all three agents, applies severity weighting, and issues a final pass/fail determination. On failure, generates actionable remediation suggestions.

The validation pipeline runs automatically on any pull request that modifies files under the `infrastructure/` directory, providing immediate feedback to developers before code review.

7.3 Test Suite

The project includes a comprehensive pytest test suite covering all pipeline stages:

Table 7.2: Test Modules and Coverage

Module	Coverage Scope	Tests
<code>test_api.py</code>	Flask endpoint correctness: health response structure, predict returns 72 hours, model selection works, historical data respects parameters	6
<code>test_data_pipeline.py</code>	Data ingestion (API mocking), async retry logic, feature engineering output shape and column presence, null handling	8
<code>test_feature_pipeline.py</code>	ClearML Dataset creation, Parquet write/read, feature store retrieval with time filtering	5
<code>test_training_pipeline.py</code>	Model training runs to completion, metrics are finite, champion selection logic, model serialization round-trip	7
<code>test_validation.py</code>	LangGraph agent execution, Rego policy evaluation, end-to-end validation pipeline pass/fail	4

Listing 7.1: Running the test suite

```
1 # Full test suite with verbose output
2 pytest tests/ -v
3
4 # With coverage report
5 pytest tests/ --cov=. --cov-report=term-missing --cov-report=html
6
7 # Run specific module
8 pytest tests/test_api.py -v --tb=short
```

All tests use mocked external dependencies (API calls, ClearML connections) to enable fast, deterministic execution without network access.

Chapter 8

Deployment and Infrastructure

8.1 Production Architecture

The production system uses a split deployment strategy optimized for the free tier of modern cloud platforms:

Table 8.1: Production Deployment Architecture

Component	Platform	Details
Frontend (Next.js)	Vercel	React 19 + TypeScript + Tailwind CSS 4, automatic deploys on git push, global CDN, serverless functions
Backend (Flask API)	Render	Python 3.11, Gunicorn with 2 workers, automatic deploys from GitHub, free tier with auto-sleep
Feature Store	ClearML	Dataset versioning with Parquet artifacts, experiment tracking, model registry
CI/CD	GitHub Actions	Hourly feature pipeline, daily training pipeline, PR validation

8.2 Frontend Deployment (Vercel)

The Next.js 16 dashboard is deployed on Vercel with the following configuration:

- **Framework:** Next.js 16 (App Router) with React 19
- **Root Directory:** deployment/web_app
- **Build Command:** npm run build
- **Environment Variable:** NEXT_PUBLIC_API_URL pointing to Render backend
- **Features:** Framer Motion animations, Plotly charts, model selector, SHAP/LIME visualization pages
- **Performance:** Edge-cached static assets, ISR for dynamic pages, Lighthouse score > 90

8.3 Backend Deployment (Render)

The Flask REST API is deployed on Render's free web service tier:

Listing 8.1: Render start command

```
1 gunicorn deployment.api.main:app --bind 0.0.0.0:$PORT --workers 2
   --timeout 120
```

- **Runtime:** Python 3.11, auto-detected from `requirements.txt`
- **Build:** `pip install -r requirements.txt`
- **Health Check:** GET `/health` endpoint (30s interval)
- **Environment:** API keys injected via Render's encrypted environment variables
- **Cold Start:** Free tier sleeps after 15 minutes of inactivity; first request takes ~30s

8.4 Docker Configuration

For local development and self-hosted deployment, a multi-stage Dockerfile optimizes for minimal image size:

Listing 8.2: Multi-stage Dockerfile architecture

```
1 # Stage 1: Builder (compiles native dependencies)
2 FROM python:3.11-slim AS builder
3 RUN apt-get install -y gcc g++ cmake
4 COPY requirements.txt .
5 RUN pip install --no-cache-dir -r requirements.txt
6
7 # Stage 2: Runtime (production-optimized)
8 FROM python:3.11-slim AS runtime
9 RUN groupadd -r appuser && useradd -r -g appuser appuser
10 COPY --from=builder /opt/venv /opt/venv
11 COPY . /app
12 USER appuser
13 EXPOSE 8000
14 CMD ["gunicorn", "deployment.api.main:app", "--bind", "
    0.0.0.0:8000",
15     "--workers", "2", "--timeout", "120"]
```

Security measures include: non-root execution, no build tools in runtime image, no secrets baked into layers, and an explicit health check probe.

8.5 Infrastructure as Code (Terraform)

For enterprise-scale deployment, Terraform configurations provision a fully serverless AWS architecture:

Table 8.2: AWS Resources (Terraform-managed)

Service	Purpose	Configuration
Lambda	Feature pipeline (hourly)	Container image, 512MB RAM, 5min timeout
Lambda	API serving (Function URL)	Container image, 1GB RAM, 30s timeout
ECS Fargate	Training pipeline (daily)	2 vCPU, 4GB RAM, spot instances
ECR	Container registry	Lifecycle: retain last 10 images
EventBridge	Scheduling	Cron expressions for hourly/daily triggers
SSM Parameter Store	Secrets management	Encrypted API keys (Secure-String)
CloudWatch	Observability	Log groups with 30-day retention, custom metrics
CodePipeline	CI/CD	Source (GitHub) → Build (CodeBuild) → Deploy

Listing 8.3: Terraform provisioning

```

1 cd infrastructure/terraform
2 terraform init          # Initialize providers and backend
3 terraform plan          # Preview resource changes
4 terraform apply         # Provision infrastructure
5 terraform output        # Display endpoint URLs and ARNs
    
```

8.6 Local Development Setup

Listing 8.4: Complete local development workflow

```

1 # 1. Clone and create virtual environment
2 git clone https://github.com/Zain-ul-abdeen-773/AQI-Predictor.git
3 cd AQI-Predictor
4 python -m venv venv && source venv/bin/activate # Linux/Mac
5 pip install -r requirements.txt
6
7 # 2. Configure environment variables
8 cp .env.example .env
9 # Edit .env with: AQICN_API_KEY, OPENWEATHER_API_KEY, CLEARML
   keys
10
11 # 3. Generate training data (fetches real historical data from
   Open-Meteo)
12 python -m data_pipeline.backfill --start-date 2022-07-28
13
14 # 4. Run exploratory data analysis
15 python -m eda.run_eda && python -m eda.advanced_eda
    
```

```
16
17 # 5. Train all 9 models
18 python -m training_pipeline.train
19
20 # 6. Launch Flask API backend
21 flask --app deployment.api.main:app run --port 8000
22
23 # 7. Launch Streamlit dashboard (separate terminal)
24 streamlit run deployment/streamlit_app/app.py
25
26 # 8. Launch Next.js frontend (separate terminal)
27 cd deployment/web_app && npm install && npm run dev
28
29 # 9. Run tests
30 pytest tests/ -v --cov=. --cov-report=term-missing
31
32 # 10. Docker deployment (alternative to steps 6-8)
33 docker-compose up --build
```

Chapter 9

Conclusion and Future Work

9.1 Project Summary

The Pearls AQI Predictor delivers a complete, production-grade MLOps solution for air quality forecasting in Sargodha, Pakistan. The system addresses a critical public health need — with 36.7% of observed hours classified as hazardous, accurate multi-day forecasting enables preventive action for vulnerable populations.

Table 9.1: Project Deliverables Summary

Component	Deliverable
Data Pipeline	Real-time ingestion (AQICN + OpenWeather) + historical back-fill (Open-Meteo), 43,801+ hourly observations
Feature Engineering	37 engineered features with leakage prevention, cyclical encoding, lag features, rolling statistics
Model Development	9 trained models spanning 4 paradigms with Optuna Bayesian HPO (50 trials)
Explainability	SHAP (TreeExplainer + GradientExplainer) + LIME (TabularExplainer) + Temporal Grad-CAM
EDA	19-stage analysis producing 20 visualizations and 26 statistical reports
Automation	Hourly ingestion + daily retraining via GitHub Actions; Lang-Graph infra validation
Deployment	Next.js frontend (Vercel) + Flask API (Render) + Docker + Terraform IaC
Testing	30+ pytest tests across 5 modules covering all pipeline stages
Documentation	40+ page LaTeX technical report + README with Mermaid architecture diagrams

9.2 Key Achievements

- Achieved $R^2 = 0.903$ (champion model) on 72-hour ahead AQI forecasting with MAPE of 9.23%

- Built a fully automated pipeline requiring **zero manual intervention** for ongoing production operation
- Demonstrated **Granger causality** linking temperature, wind speed, and pressure to AQI — validating the scientific basis for using meteorological features
- Identified **thermal inversions** (low wind + cool temperatures) as the primary driver of hazardous events in Sargodha through comprehensive EDA
- Implemented **three complementary explainability methods** providing both global and local model interpretability at feature and temporal levels
- Deployed a **production-ready system** with interactive Next.js dashboard, REST API, and automated CI/CD

9.3 Limitations

- Training data availability limited by Open-Meteo API coverage (historical data from 2022 onwards)
- LightGBM/XGBoost Optuna HPO requires sufficient training samples; small datasets may cause convergence issues
- Free-tier deployment (Render) introduces cold-start latency (~30s after 15 minutes of inactivity)
- Model assumes stationarity of pollution sources; structural changes require retraining
- Current confidence intervals use heuristic widening; formal conformal prediction would provide theoretical guarantees

9.4 Future Work

1. **Multi-City Expansion:** Extend to Lahore, Karachi, Islamabad, and Faisalabad with city-specific fine-tuned models
2. **Satellite Integration:** Incorporate MODIS Aerosol Optical Depth (AOD) and Sentinel-5P NO₂ column data for spatial pollution mapping
3. **Ensemble Stacking:** Combine top-3 models via a meta-learner for improved predictions beyond any individual model
4. **Online Learning:** Implement incremental updates using river/Vowpal Wabbit to adapt without full retraining
5. **Mobile Alerts:** Firebase Cloud Messaging push notifications for hazardous AQI predictions targeting sensitive groups
6. **Causal Discovery:** Apply PC algorithm or NOTEARS for causal graph structure learning between pollution sources

7. **Conformal Prediction:** Replace heuristic confidence intervals with distribution-free coverage guarantees
8. **Model Monitoring:** Integrate Evidently AI for automated data drift and model degradation detection in production

Appendix A

Environment Variables

Table A.1: Complete Environment Variable Reference

Variable	Default	Description
AQICN_API_KEY	demo	AQICN API token
OPENWEATHER_API_KEY	–	OpenWeatherMap key
CLEARML_API_ACCESS_KEY	–	ClearML access key
CLEARML_API_SECRET_KEY	–	ClearML secret key
TARGET_CITY	Sargodha	Forecast target
FORECAST_HORIZON_HOURS	72	Prediction window
LOOKBACK_WINDOW_HOURS	72	Input sequence length
AQI_ALERT_THRESHOLD	150	Alert trigger level
LSTM_HIDDEN_SIZE	128	LSTM hidden dim
LSTM_NUM_LAYERS	2	LSTM layers
OPTUNA_N_TRIALS	50	HPO budget
LOG_LEVEL	INFO	Logging verbosity

Appendix B

Project File Structure

Listing B.1: Complete project structure

```
1 .
2 +-- config/           Configuration (Pydantic BaseSettings)
3 +-- eda/              Exploratory Data Analysis (19 analyses)
4 +-- data_pipeline/    Data ingestion & feature engineering
5 +-- feature_pipeline/ ClearML Feature Store management
6 +-- training_pipeline/ Model training, evaluation,
   explainability
7 |   +-- models/       9 model implementations
8 +-- deployment/
9 |   +-- api/          Flask REST API
10 |   +-- streamlit_app/ Streamlit dashboard
11 +-- infrastructure/
12 |   +-- terraform/    AWS IaC
13 |   +-- validation_agents/ LangGraph multi-agent validation
14 +-- data/             Feature store (Parquet) + backfill CSV
15 +-- models/           Trained model artifacts
16 +-- tests/            pytest test suite
17 +-- docs/             This documentation
18 +-- .github/workflows/ CI/CD (hourly + daily + PR)
19 +-- Dockerfile        Multi-stage build
20 +-- docker-compose.yml Container orchestration
21 +-- requirements.txt   Python dependencies
```